

Überladen und Überschreiben

Aspekt	Überladen (Overloading)	Überschreiben (Overriding)
Definition	Das Definieren mehrerer Methoden mit dem gleichen Namen, aber unterschiedlicher Parameterliste in derselben Klasse.	Das Ersetzen einer Methode in einer Unterklasse, die in der Oberklasse mit derselben Signatur definiert wurde.
Hauptzweck	Bietet Flexibilität durch Methoden mit demselben Namen, aber unterschiedlichen Parametern.	Ermöglicht spezifisches Verhalten in der Unterklasse für eine Methode der Oberklasse.
Ort	Nur in derselben Klasse oder zwischen einer Klasse und ihrer ererbenden Klasse.	Zwischen einer Oberklasse und einer ihrer Unterklassen.
Parameterliste	Muss sich in Anzahl, Typ oder Reihenfolge der Parameter unterscheiden.	Muss exakt identisch sein, einschließlich Anzahl, Typ und Reihenfolge der Parameter.
Rückgabotyp	Kann unterschiedlich sein, da die Parameterliste zur Differenzierung verwendet wird.	Muss identisch sein oder ein Subtyp des Rückgabetyps der überschriebenen Methode (kovarianter Rückgabewert).
Zugriffsmodifizierer	Kann sich zwischen Methoden unterscheiden.	Darf nicht restriktiver sein als in der Oberklasse. Beispiele: <code>protected</code> in der Oberklasse -> mindestens <code>protected</code> in der Unterklasse.
Statische Methoden	Erlaubt, da sie keine Instanzbindung haben.	Nicht möglich – eine statische Methode kann nicht überschrieben, sondern nur versteckt werden ("method hiding").
Final-Methoden	Können überladen werden, da die Signatur neu ist.	Nicht erlaubt, da <code>final</code> Methoden in der Oberklasse vor Überschreibung schützt.
Konstruktoren	Können überladen werden (mehrere Konstruktoren in derselben Klasse).	Konstruktoren können nicht überschrieben werden, da sie nicht vererbbar sind.

Aspekt	Überladen (Overloading)	Überschreiben (Overriding)
Annotationsunterstützung	Keine Annotation erforderlich.	Erfordert <code>@override</code> (optional, aber empfohlen zur Vermeidung von Fehlern).
Beziehung zur Laufzeitbindung	Kein Einfluss, da die Methode zur Compilezeit gebunden wird (statische Bindung).	Einfluss, da die Methode zur Laufzeit gebunden wird (dynamische Bindung).
Ausnahmebehandlung	Kann verschiedene Ausnahmen werfen, unabhängig von der Basis-Methode.	Kann dieselben oder weniger restriktive Ausnahmen werfen.
Leistungsunterschied	Wird zur Compilezeit aufgelöst und hat keinen Einfluss auf die Laufzeitleistung.	Wird zur Laufzeit basierend auf der tatsächlichen Objektreferenz aufgelöst, was minimale Laufzeitkosten hat.
Kompatibilität mit Polymorphie	Wird nicht durch Polymorphie unterstützt, da die Methode anhand der Parameterliste zur Compilezeit ausgewählt wird.	Unterstützt Polymorphie, da die Methode basierend auf dem Objekttyp zur Laufzeit aufgerufen wird.